ARTEFACT

Challenge Data Engineer

Level 3 - Pipeline hybride Batch et Streaming

Guide ultra simplifié et détaillé : comprendre exactement ce qui est attendu, avant de coder.

Objectif de lecture
À la fin, tu dois pouvoir expliquer sans jargon pourquoi la plateforme garde le batch du Level 2, ce que Kafka et NiFi ajoutent, comment Spark Streaming traite un événement, et comment la fraude ou l'AML deviennent des alertes exploitables.

Périmètre : données fictives WABA, huit pays, banque, assurance, mobile money et microfinance.

Ce guide distingue les exigences écrites du challenge, les choix d'implémentation à documenter et les limites normales d'un prototype local.

1. Ce que le Level 3 te demande réellement

Le Level 3 étend la plateforme des Levels 1 et 2. Il ne remplace pas le batch : il ajoute une voie de traitement presque temps réel pour des événements financiers fraîchement produits.

Le besoin métier

Dans une banque ou une assurance, attendre le batch de nuit est parfois trop tard. Une transaction inhabituelle, un virement très élevé ou un risque de liquidité doivent être repérés pendant que l'activité se déroule.

Question métier	Réponse apportée par le streaming
Une transaction peut-elle être frauduleuse ?	On analyse les événements en quelques secondes ou minutes et on publie une alerte.
Un virement dépasse-t-il le seuil AML ?	On détecte le dépassement dès l'arrivée de l'événement.
Que se passe-t-il maintenant ?	Kafka fournit les événements récents ; le batch reste la vue historique consolidée.

Exigence explicite
Mettre en place une architecture Lambda : batch + streaming. Les prérequis indiquent que le Level 2 doit être fonctionnel.

Le résultat attendu

- NiFi détecte de nouveaux fichiers financiers dans MinIO et publie leurs lignes dans Kafka.
- Kafka contient des topics Raw, Silver et Gold, plus une Dead Letter Queue (DLQ).
- Deux jobs Spark Structured Streaming transforment les événements et produisent des alertes.
- Trino peut interroger ensemble les données historiques Iceberg et les données récentes Kafka.

2. L'intuition : un même événement, deux vitesses

Un événement est un fait qui vient de se produire : un virement, un paiement mobile, un remboursement de prêt ou une opération d'assurance. Dans le Level 1, ces faits sont regroupés dans un CSV. Dans le Level 3, chaque ligne de ce CSV devient aussi un message individuel.

Le batch, déjà construit

Le batch prend un ou plusieurs fichiers CSV, les valide, puis les consolide dans Bronze, Silver et Gold. Il est très adapté aux recalculs complets, aux rapports consolidés et aux KPI historiques.

Le streaming, ajouté au Level 3

Le streaming ne cherche pas à attendre un fichier complet de la journée. Il traite les nouvelles lignes au fil de leur arrivée. Spark Structured Streaming travaille souvent par micro-lots : de très petits lots successifs, mais de manière continue.

Point essentiel
Le même fichier peut alimenter la voie batch et la voie streaming. C'est voulu : la voie batch donne une vérité historique recalculable ; la voie streaming donne une vision fraîche. C'est le pattern Lambda.

3. Architecture Lambda : comment relier les Levels 1, 2 et 3

Lambda signifie ici : deux chemins de traitement complémentaires qui partent des mêmes données brutes. Le challenge ne demande pas de supprimer ce qui existe déjà ; il demande de l'étendre.

Couche	Entrée	Sortie	Rôle
Batch	CSV MinIO	Iceberg Bronze / Silver / Gold	Historique, qualité, consolidation et KPI.
Speed / streaming	Événements Kafka Raw	Kafka Silver + Iceberg Silver	Données fraîches validées et enrichies.
Serving	Iceberg + Kafka	Trino	Une même requête peut juxtaposer historique et fraîcheur.

Pourquoi il faut conserver les deux chemins

- Le batch peut recalculer tout un trimestre si une règle métier ou un taux de change change.
- Le streaming réduit le délai entre un événement et une alerte.
- Iceberg garde les données structurées et historisées ; Kafka garde le flux récent d'événements.
- Trino devient le point d'accès SQL commun pour la consommation analytique.

Décision d'architecture à documenter
Le déplacement d'un CSV de raw-landing vers archive doit attendre que les consommateurs nécessaires aient fini. Sinon NiFi pourrait manquer un fichier archivé trop tôt. Ce sujet de cycle de vie des fichiers se traite avec soin, sans casser la voie batch existante.

4. Kafka sans jargon

Kafka est un journal distribué d'événements. Il ne transforme pas les données : il les reçoit, les conserve un temps, puis permet à plusieurs applications de les lire indépendamment.

Terme	Intuition simple	Dans WABA
Broker	Le serveur Kafka qui stocke et distribue les messages.	Le conteneur Kafka de la plateforme.
Topic	Un canal nommé pour une famille d'événements.	raw-bank-transactions ou gold-fraud-alerts.
Producteur	Une application qui écrit dans un topic.	NiFi écrit les messages Raw.
Consommateur	Une application qui lit un topic.	Spark Streaming lit les messages Raw ou Silver.
Consumer group	Une équipe de consommateurs qui se partage le travail.	Un job Spark Streaming possède son groupe.
Offset	Le numéro de la dernière ligne Kafka lue.	Après redémarrage, Spark peut reprendre.

ZooKeeper ou KRaft

Le challenge accepte les deux. ZooKeeper est le composant de coordination historique de Kafka : il aide notamment à connaître les brokers et les leaders. KRaft est le mode moderne qui intègre cette coordination dans Kafka.

> **Décision possible pour ce projet**
> Ta configuration Kafka existante avec ZooKeeper est conforme à l'énoncé. Il n'est pas nécessaire de migrer vers KRaft si la stack est stable, claire et documentée.

5. Les topics attendus par le challenge

Un topic sépare les familles d'événements. Cela évite de mélanger un paiement mobile, un sinistre et un remboursement de prêt dans le même flux.

Topic	Couche	Contenu
raw-bank-transactions	Raw	Transactions bancaires brutes publiées par NiFi.
raw-insurance-operations	Raw	Opérations d'assurance brutes publiées par NiFi.
raw-mobile-money-payments	Raw	Paiements mobile money bruts publiés par NiFi.
raw-loan-repayments	Raw	Remboursements de prêt bruts publiés par NiFi.
silver-bank-transactions	Silver	Transactions validées et enrichies par Spark Streaming Job 1.
silver-insurance-operations	Silver	Opérations assurance enrichies par Spark Streaming Job 1.
silver-mobile-money	Silver	Paiements mobile money enrichis par Spark Streaming Job 1.
gold-fraud-alerts	Gold	Alertes fraude détectées par Spark Streaming Job 2.
gold-aml-events	Gold	Événements suspects AML dépassant un seuil réglementaire.
gold-liquidity-alerts	Gold	Alertes de liquidité agrégées par pays.
dlq-financial-events	DLQ	Messages invalides, conservés pour investigation.

Pourquoi une DLQ ?
Un message invalide ne doit ni faire tomber tout le flux ni disparaître silencieusement. Il est envoyé dans dlq-financial-events avec sa raison de rejet ; l'équipe data peut l'analyser et corriger sa source.

6. NiFi : le pont entre MinIO et Kafka

NiFi est l'outil de collecte et de routage. Dans ce challenge, il surveille raw-landing, lit les nouveaux fichiers et publie leurs lignes dans les topics Kafka adaptés.

Ce que NiFi doit faire

- Surveiller MinIO avec les processeurs indiqués : ListS3 puis FetchS3Object.
- Lire un CSV ; pour chaque ligne, produire un événement JSON.
- Choisir le topic selon le type de données et le pays.
- Ajouter ingestion_timestamp et source_file avant publication.
- Configurer une back-pressure : NiFi ralentit la lecture si Kafka est saturé, au lieu de consommer toute la mémoire.

Exemple mental

Le fichier `CI/bank_transactions/bank_txn_CI_20261004_01.csv` contient 10 000 lignes. NiFi ne publie pas le fichier comme un seul message ; il publie 10 000 événements JSON dans `raw-bank-transactions`. Chaque message conserve le pays, le type, le fichier source et la date d'ingestion.

> **Important**
> NiFi ne remplace pas le générateur Streamlit. Streamlit simule les systèmes métiers et produit les CSV ; NiFi simule une collecte temps réel qui transforme ces fichiers en événements de streaming.

7. Spark Structured Streaming - Job 1 : Raw vers Silver

Le premier job est le gardien de qualité du flux temps réel. Il lit Kafka, vérifie les messages, applique les transformations Silver et écrit deux sorties.

Étapes demandées

Étape	Ce que fait le job	Pourquoi
1. Consommer	Lit les topics raw-* depuis Kafka.	Récupérer les nouveaux événements.
2. Valider	Contrôle le schéma JSON et les champs attendus.	Éviter des données corrompues dans Silver.
3. DLQ	Envoie les messages invalides dans dlq-financial-events.	Préserver la trace et continuer le flux.
4. Nettoyer	Cast des types, dates et montants ; contrôle de devise.	Normaliser les données.
5. Dédupliquer	Déduplique dans une fenêtre de 10 minutes selon l'identifiant métier.	Gérer les rediffusions ou doublons proches.
6. Convertir	Convertit les montants en EUR pour les analyses multi-pays.	Comparer XOF et GHS.

Identifiant de déduplication

Le challenge cite `transaction_id`. Le même principe s'applique naturellement à `operation_id`, `payment_id` et `repayment_id`, car chaque flux possède sa clé technique.

8. Le double sink : deux sorties pour le Job 1

Une fois un événement validé et enrichi, le Job 1 doit l'écrire à la fois dans Kafka Silver et dans des tables Iceberg Silver. C'est le double sink demandé.

Pourquoi ne pas écrire seulement dans Kafka ?

- Kafka est excellent pour transporter des événements récents, mais ce n'est pas le stockage analytique historique principal.
- Iceberg permet la conservation, le partitionnement, les snapshots et les requêtes SQL sur un historique.
- Kafka Silver alimente immédiatement le Job 2 de fraude ; Iceberg Silver reste la base consolidée des analyses.

Point technique à expliquer en entretien

Un job streaming doit conserver son état de reprise dans un checkpoint. Le checkpoint permet de savoir jusqu'où Kafka a été lu et de reprendre après un redémarrage. C'est une décision technique nécessaire au fonctionnement robuste, même si l'énoncé ne cite pas ce mot.

Limite honnête du prototype
La vraie garantie exactement-une-fois entre deux systèmes différents est avancée. Ici, on vise un comportement démontrable : checkpoints, clés métier, déduplication et écritures idempotentes/documentées.

9. Spark Structured Streaming - Job 2 : fraude, AML et liquidité

Le second job lit les événements Silver. Son objectif n'est plus de nettoyer les données : il applique des règles métier de surveillance et publie des alertes Gold.

Cas	Règle demandée	Sortie
Fraude bancaire	Plusieurs transactions supérieures à 500 000 XOF depuis le même compte en moins de 5 minutes.	gold-fraud-alerts
Fraude mobile	Paiements mobile money depuis des pays inhabituels pour le profil client.	gold-fraud-alerts
Fraude assurance	Montant d'un sinistre supérieur à trois fois la prime annuelle versée.	gold-fraud-alerts
AML	Virement au-dessus du seuil BCEAO : 1 000 000 XOF en UEMOA ; 5 000 GHS au Ghana.	gold-aml-events
Liquidité	Surveiller en continu les soldes agrégés par pays et alerter si un seuil de couverture est franchi.	gold-liquidity-alerts

Fenêtres glissantes

Pour la fraude bancaire, le challenge demande une fenêtre de 5 minutes qui avance toutes les 1 minute. À chaque minute, Spark regarde les cinq dernières minutes. C'est ce qui permet de détecter une série suspecte, pas seulement une transaction isolée.

> **Attention métier**
> Une alerte n'accuse pas un client de fraude. Elle signale un événement ou un comportement à examiner par une équipe risque, conformité ou fraude.

10. AML et liquidité : deux surveillances différentes

AML : Anti-Money Laundering

AML signifie lutte contre le blanchiment d'argent. Dans le challenge, une règle simple de seuil est demandée. Un virement très élevé est publié dans `gold-aml-events` afin qu'il soit examiné selon le processus de conformité.

Zone	Devise	Seuil déclaratif
Pays UEMOA : CI, SN, ML, BF, GN, TG, BJ	XOF	1 000 000 XOF
Ghana : GH	GHS	5 000 GHS

Liquidité

La liquidité répond à une autre question : l'entité dispose-t-elle de suffisamment de fonds disponibles pour honorer ses engagements ? Le challenge demande de surveiller des soldes agrégés par pays et de créer une alerte lorsque le seuil minimal de couverture est franchi.

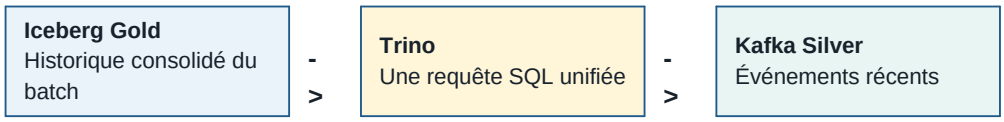
Décision à expliciter
Le schéma fourni contient des soldes de comptes dans accounts, mais pas une formule complète de couverture de liquidité. Le seuil et la méthode de calcul devront être documentés comme une hypothèse de prototype, jamais présentés comme une norme BCEAO complète.

11. Trino : rendre la vue Lambda interrogeable

Trino n'effectue pas les calculs streaming. Son rôle est de permettre à un analyste de poser une requête SQL sur les tables Iceberg historiques et sur des événements récents exposés via un connecteur Kafka.

Exemple d'intuition

Une table Gold Iceberg peut contenir le volume bancaire déjà consolidé jusqu'au dernier batch. Le topic Kafka Silver contient les transactions arrivées depuis. Une requête Lambda additionne ou joint ces deux sources pour obtenir une vue plus fraîche.



Ce qui est demandé

- Configurer Trino pour interroger les données historiques Iceberg Gold.
- Configurer aussi un connecteur Kafka pour les données fraîches.
- Prouver une requête SQL qui utilise les deux sources sans erreur.

À ne pas confondre
Kafka contient des événements ; Iceberg contient des tables analytiques. Trino ne les transforme pas en une seule table magique : il sait les lire dans une même requête SQL.

12. Erreurs, reprises et observabilité

En streaming, une erreur ne doit pas faire perdre les événements. Le challenge attend une gestion visible des messages invalides et du retard de consommation.

Risque	Réponse simple et démontrable
JSON ou schéma invalide	Envoyer le message dans dlq-financial-events avec la raison du rejet.
Kafka temporairement lent	Configurer la back-pressure NiFi pour ralentir la source.
Spark redémarre	Utiliser un checkpoint ; Spark reprend à partir des offsets connus.
Même événement rediffusé	Dédupliquer dans la fenêtre demandée avec la clé métier.
Consommateur en retard	Suivre le lag Kafka : différence entre messages produits et messages lus.

Le lag Kafka

Le lag est le nombre de messages encore non lus par un consommateur. Un lag faible signifie que le pipeline suit le flux. Le critère du challenge parle d'un lag maîtrisé, inférieur à 30 secondes pour NiFi vers les topics Raw lors de la démonstration.

> **Lien avec le Level 4**
> Les métriques Kafka, la latence Spark et les alertes Grafana sont surtout approfondies au Level 4. Au Level 3, tu dois déjà éviter les pertes silencieuses et démontrer la DLQ.

13. Critères d'évaluation Level 3 traduits simplement

Critère	Ce que l'évaluateur veut voir
NiFi -> Kafka opérationnel	Les événements arrivent dans les quatre topics raw-* ; délai raisonnable, idéalement moins de 30 secondes.
Job 1 Streaming	Les topics silver-* sont alimentés et les tables Iceberg Silver sont mises à jour.
Fraude	Les trois règles de fraude produisent des alertes dans gold-fraud-alerts avec des données simulées.
AML	Les dépassements des seuils XOF/GHS sont publiés dans gold-aml-events.
DLQ	Un message malformé arrive dans dlq-financial-events ; il n'est pas ignoré.
Trino Lambda	Une requête joint ou combine Iceberg et Kafka sans erreur.

Ce qui fait gagner des points
Ne pas seulement avoir des conteneurs démarrés. Montrer un petit scénario de bout en bout : fichier -> événement Raw -> Silver -> alerte ou DLQ -> requête Trino. Une démonstration courte et cohérente vaut mieux qu'une stack très large non vérifiée.

14. Ordre de construction recommandé

L'ordre limite les problèmes. On valide un maillon avant d'ajouter le suivant.

Étape	Petit objectif de preuve
1. Kafka + ZooKeeper	Le broker est healthy ; un producteur et un consommateur échangent un message.
2. Topics	Les 11 topics demandés existent et leur nom est conforme.
3. NiFi	Un CSV MinIO génère des messages JSON dans un topic Raw.
4. Job Streaming 1	Un message Raw valide va dans Kafka Silver et Iceberg Silver ; un invalide va en DLQ.
5. Job Streaming 2	Un événement simulé déclenche une alerte fraude puis une alerte AML.
6. Trino Kafka	Une requête SQL lit une table Iceberg et un topic Kafka.
7. Démonstration	Captures, logs et commandes de reproduction ajoutés au README.

Ce qu'il ne faut pas faire tout de suite

- Ne pas essayer d'écrire les deux jobs streaming, NiFi et Trino Kafka en une seule fois.
- Ne pas inventer de règles réglementaires non présentes dans l'énoncé sans les présenter comme hypothèses.
- Ne pas masquer un message invalide : la DLQ est précisément attendue.

15. Glossaire de restitution rapide

Terme	Phrase simple à dire en entretien
Architecture Lambda	Je garde le batch pour l'historique consolidé et j'ajoute le streaming pour la fraîcheur des événements.
Kafka	C'est le bus qui transporte et conserve temporairement les événements entre les applications.
NiFi	C'est le collecteur : il détecte les nouveaux fichiers MinIO et publie chaque ligne dans Kafka.
Spark Structured Streaming	C'est Spark appliqué à un flux continu, souvent traité par petits micro-lots.
Topic	Un canal Kafka réservé à une famille d'événements.
DLQ	Un topic de quarantaine qui garde les messages invalides et leur raison de rejet.
Checkpoint	L'état de reprise du job streaming pour éviter de perdre sa position dans Kafka.
Lag	Le retard entre les messages produits et les messages que le consommateur a déjà lus.
AML	Une surveillance des transactions potentiellement liées au blanchiment ; ici, un seuil déclenche un événement à examiner.

Phrase de synthèse
Le Level 3 transforme les CSV fictifs déjà générés en événements exploitables presque en temps réel. NiFi les publie dans Kafka, Spark Streaming les fiabilise puis détecte les risques, et Trino peut combiner cette fraîcheur avec l'historique Iceberg du batch.

Fin du guide